Experiment 02: Version Control Operations using Git

Experiment No.	02
Mapped CO	CO2
Aim	To perform fundamental and collaborative version control operations using Git and GitHub.
Tools / Software	Git (>= 2.30); A GitHub account; A code editor / terminal

Theory

Git is a distributed version control system in which every clone is a complete repository with full history. A Git project has three logical areas: the working directory (files on disk), the staging area or index (changes marked for the next commit), and the local repository (the committed history of snapshots). A commit is an immutable snapshot identified by a SHA-1/SHA-256 hash and linked to its parent, forming a directed acyclic graph.

Branches are lightweight movable pointers to commits and enable parallel lines of development. Merging integrates one branch into another; when two branches change the same lines, a merge conflict occurs and must be resolved manually. Collaboration workflows such as feature branching and Git Flow standardize how branches are created and integrated, and remotes (for example origin on GitHub) synchronize work between developers through push and pull.

Procedure

1. Configure your identity with git config (name and email).
2. Initialize a repository with git init and check status with git status.
3. Create a file, stage it with git add, and record it with git commit.
4. Create and switch to a feature branch with git switch -c feature/x.
5. Make commits on the branch, then merge back into main with git merge.
6. Deliberately edit the same line on two branches to create a conflict, then resolve it and commit.
7. Create a repository on GitHub, add it as a remote, and push with git push -u origin main.
8. Clone the repository elsewhere, make a change, push, then synchronize with git pull.

Sample Code / Commands

Core Git workflow

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git init
git add .
git commit -m "Initial commit"
git switch -c feature/login
git add . && git commit -m "Add login"
git switch main
git merge feature/login
git remote add origin https://github.com/<user>/<repo>.git
git push -u origin main
```

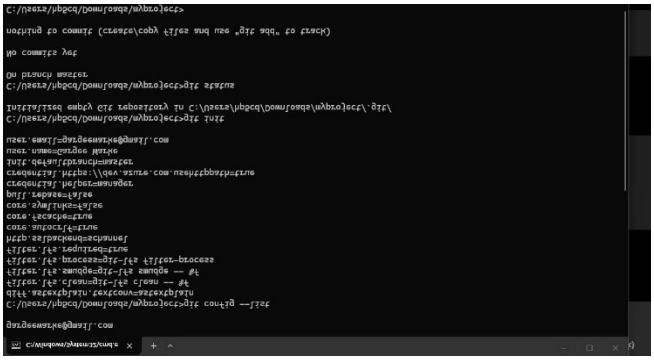
Observation / Experiment Output

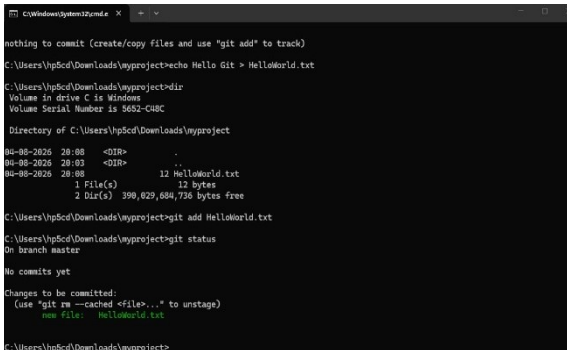
Instructions to students: Paste screenshots or copied terminal outputs in the boxes below after completing the Git commands. Keep screenshots clear and readable.

Name	Roll No.	Batch	Date
Sagar Pradhan	49	B2	31-07-26

1. Terminal Output / Command Output

Paste output for: git status, git branch, git log --oneline --graph, git remote -v, git push -u origin main

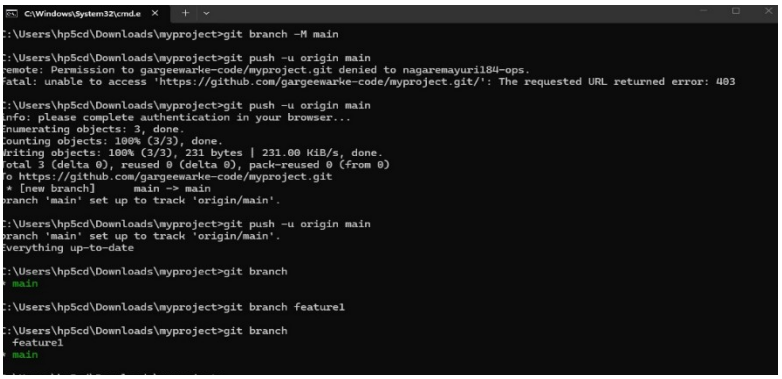




Paste output / screenshot here

2. Branching and Merge Output

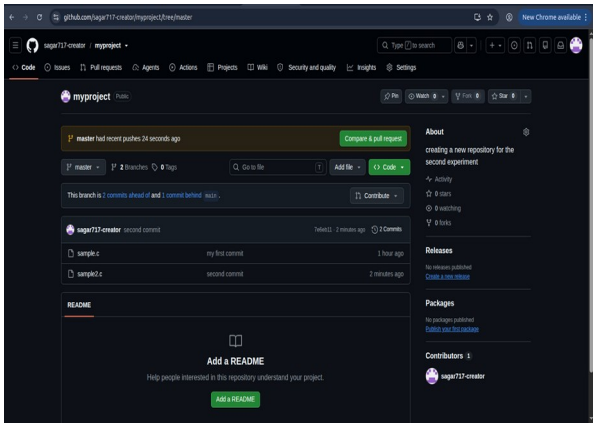
Paste screenshot/output showing feature branch creation, commit on branch, switching to main, and merging the branch.

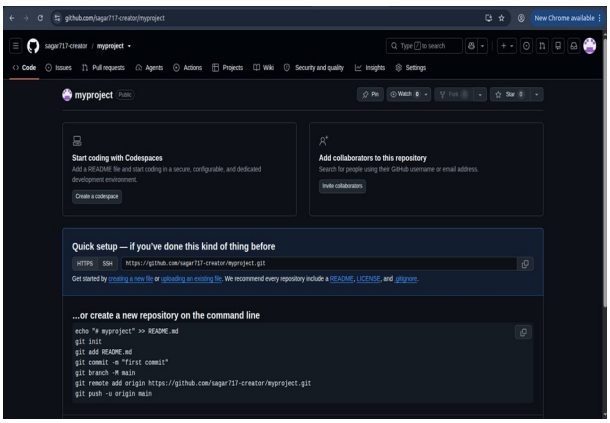


3. Merge Conflict and Resolution Output



4. GitHub Repository Output





GitHub Repository Link: https://github.com/sagar717-creator/myproject

Final Observation Statement:

In this experiment, I learned how to use Git and GitHub for version control. I successfully created a repository, added and committed files, created and merged branches, resolved a merge conflict, and synchronized my local project with GitHub using push and pull commands.

Expected Output

A repository with a multi-commit history visible via git log --oneline --graph, at least one merged feature branch, a resolved merge conflict, and the local history mirrored on GitHub after push.

Conclusion

Local and collaborative Git operations, including branching, merging, conflict resolution and remote synchronization, were performed successfully.

Viva / Review Questions

1. Distinguish between git fetch and git pull.
2. What is the difference between git merge and git rebase?
3. Explain the three states of a file in Git.
4. What is the purpose of a .gitignore file?

Marks: ____ / 10 Date: _____ Signature of Faculty: _____